

Análise Comparativa de Desempenho de Algoritmos de Ordenação em Diferentes Cenários de Entrada

Felipe Rochoel Costa

Graduação em Engenharia de Computação
Universidade Tecnológica Federal do Paraná (UTFPR) – Campus Apucarana
Apucarana – PR – Brasil
`feliperochoelcosta@alunos.utfpr.edu.br`

Abstract. This paper presents an empirical and comparative performance analysis of three widely studied sorting algorithms: Bubble Sort, Merge Sort, and Quick Sort. The evaluation focuses on execution time and the number of key comparisons and element swaps across three initial data ordering scenarios (random, inversely sorted, and nearly sorted lists) with dataset sizes ranging from 10,000 to 1,000,000 elements. The experimental results validate the theoretical asymptotic complexities, highlights the overhead of quadratic algorithms in large volumes of data, and explores the practical efficiency trade-offs between divide-and-conquer approaches. Source code and experimental datasets are publicly available.

Resumo. Este artigo apresenta uma análise empírica e comparativa de desempenho de três algoritmos de ordenação amplamente estudados: Bubble Sort, Merge Sort e Quick Sort. A avaliação centra-se no tempo de execução e no número de comparações de chaves e trocas de elementos sob três cenários de ordenação inicial dos dados (listas aleatórias, inversamente ordenadas e quase ordenadas), com tamanhos de conjuntos de dados variando de 10.000 a 1.000.000 de elementos. Os resultados experimentais validam as complexidades assintóticas teóricas, evidenciam o gargalo de algoritmos quadráticos em grandes volumes de dados e exploram as compensações de eficiência prática entre as abordagens de divisão e conquista. O código-fonte e os dados experimentais estão disponíveis publicamente.

1 Introdução

A ordenação de dados constitui um dos problemas mais fundamentais e exaustivamente explorados no âmbito da Ciência da Computação e da Engenharia de Software. Definida como o processo de arrumar uma coleção de registros em uma sequência linear determinada por uma relação de ordem total, a ordenação eficiente é um requisito crítico para a

otimização de operações subsequentes, tais como buscas binárias, eliminação de duplicatas e processamento de bancos de dados.

Com o advento do *Big Data* e o crescimento exponencial do volume de informações manipuladas por sistemas computacionais modernos, a escolha do algoritmo de ordenação ideal deixou de ser uma mera decisão conceitual para tornar-se um fator determinante no consumo de recursos, na latência e na escalabilidade de softwares. Diferentes algoritmos apresentam comportamentos distintos que dependem diretamente não apenas do número de elementos a ordenar, mas também do estado de ordenação prévio desse conjunto de dados.

Este trabalho tem como objetivo central realizar um estudo comparativo e experimental de três algoritmos de ordenação proeminentes: o Bubble Sort, o Merge Sort e o Quick Sort. A análise avalia o impacto do tamanho da entrada de dados (considerando vetores de tamanho 10.000, 100.000 e 1.000.000) e o comportamento sob diferentes distribuições iniciais dos elementos (listas aleatórias, inversamente ordenadas e quase ordenadas). Por meio da mensuração do tempo computacional de execução e da contagem de operações primitivas (comparações e trocas), busca-se confrontar os achados práticos com as predições matemáticas da análise assintótica clássica.

2 Fundamentação Teórica

A eficiência de um algoritmo é formalmente avaliada por meio da notação assintótica grande-O (O), que descreve o limite superior do crescimento do tempo de execução em função do tamanho da entrada (n). Os algoritmos escolhidos para esta pesquisa pertencem a duas classes principais de complexidade e estratégias de projeto distintas.

2.1 O Algoritmo Bubble Sort e sua Complexidade

O Bubble Sort (ou ordenação por bolha) é um algoritmo elementar essencialmente iterativo e baseado em comparações. O princípio operacional consiste em percorrer o vetor repetidas vezes a partir do início, comparando pares de elementos adjacentes. Caso dois elementos consecutivos estejam fora da ordem desejada, realiza-se uma operação de troca (*swap*). Esse procedimento faz com que os maiores elementos “flutuem” gradualmente para as posições finais, assemelhando-se ao movimento de bolhas em um fluido.

Do ponto de vista analítico, o Bubble Sort possui uma complexidade de tempo de pior caso e caso médio avaliada em $O(n^2)$. O pior caso ocorre quando o vetor de entrada encontra-se ordenado em ordem estritamente inversa, exigindo $n(n-1)/2$ comparações e o mesmo número de trocas. Se implementado com uma otimização baseada em uma variável sinalizadora (*flag*) para interromper a execução caso nenhuma troca ocorra em uma passagem completa, o algoritmo atinge uma complexidade de melhor caso de $O(n)$, aplicável quando o vetor já está ordenado. Sua complexidade de espaço é estável em $O(1)$, classificando-o como um algoritmo *in-place*. Apesar de didaticamente difundido e simples de codificar, sua eficiência degrada de maneira proibitiva em conjuntos de dados massivos.

2.2 O Algoritmo Merge Sort e sua Complexidade

O Merge Sort é um algoritmo de ordenação estável fundado no paradigma de projeto de “dividir para conquistar”, proposto por John von Neumann em 1945. O funcionamento do algoritmo divide-se em três etapas recursivas:

1. **Divisão:** O vetor original é segmentado em dois subvetores de tamanho aproximado $n/2$.
2. **Conquista:** Cada subvetor é ordenado recursivamente por meio de novas chamadas ao Merge Sort. Quando um subvetor atinge tamanho unitário, ele está trivialmente ordenado.
3. **Combinação (Merge):** Os subvetores ordenados são intercalados de forma linear para recompor um único vetor totalmente ordenado.

A propriedade mais notável do Merge Sort é a sua consistência temporal: ele garante uma complexidade de tempo de $O(n \log n)$ para o melhor, pior e médio caso, visto que a árvore de recorrência sempre mantém profundidade $\log n$ e o custo de intercalação em cada nível é $O(n)$. No entanto, essa estabilidade possui um custo espacial relevante. A operação de intercalação convencional não é realizada *in-place*, requerendo a alocação de um vetor auxiliar temporário de tamanho $O(n)$, o que pode limitar sua viabilidade em sistemas com restrição severa de memória RAM.

2.3 O Algoritmo Quick Sort e sua Complexidade

O Quick Sort, idealizado por C. A. R. Hoare em 1959, também emprega o paradigma de divisão e conquista, porém de maneira distinta do Merge Sort. A ordenação é estruturada em torno de um elemento de referência denominado *pivô*. A operação crítica do algoritmo é o particionamento, que rearranja os elementos do vetor de tal modo que todos os itens estritamente menores que o pivô sejam posicionados à sua esquerda, e todos os elementos maiores ou iguais fiquem dispostos à sua direita. Após o particionamento, o pivô ocupa sua posição definitiva e o processo é aplicado recursivamente nos dois subvetores resultantes.

Em cenários médios e favoráveis, onde o pivô divide o vetor em duas partes razoavelmente equilibradas, o Quick Sort exibe complexidade de tempo de $O(n \log n)$. Devido a fatores como uma constante multiplicativa baixa e excelente localidade de referência para caches de CPU, ele costuma superar o Merge Sort na prática. Contudo, a eficiência do Quick Sort é altamente sensível à escolha do pivô. No pior caso — que se manifesta quando o pivô escolhido é sistematicamente o menor ou o maior elemento do conjunto (comum em estruturas preexistentes de ordenação se fixado no primeiro elemento) — a árvore de chamadas recursivas atinge profundidade n , degradando a complexidade para $O(n^2)$. O consumo de espaço na pilha de execução varia de $O(\log n)$ no caso ideal a $O(n)$ no pior cenário.

3 Método e Procedimentos

Para coletar as métricas necessárias à comparação prática dos algoritmos descritos, foi desenvolvido um ambiente de experimentação controlado.

3.1 Ambiente de Testes

Os algoritmos foram implementados de forma homogênea utilizando a linguagem de programação C. Os experimentos computacionais foram executados de maneira isolada em uma máquina cujas especificações de hardware e software estão detalhadas a seguir:

- **Processador:** AMD Ryzen 5 5600G (3.90 GHz)
- **Memória RAM:** 16 GB DDR4
- **Sistema Operacional:** Windows 11 Pro (64 bits, versão 24H2)
- **Compilador / Ambiente de Execução:** GCC (MinGW) utilizando o editor Visual Studio Code (VS Code).

A measurement do tempo de execução foi realizada por meio de temporizadores de alta precisão da biblioteca `<time.h>`, capturando o intervalo exato entre o início e o término da rotina de ordenação. Para quantificar o esforço computacional independente de hardware, contadores internos foram inseridos no código-fonte para computar de forma exata o número de comparações de chaves efetuadas e a quantidade de trocas de posição executadas.

Visando garantir a total transparência, auditabilidade acadêmica e a reprodutibilidade integral dos experimentos práticos aqui apresentados, o código-fonte desenvolvido, os roteiros de testes e os conjuntos completos de dados textuais brutos foram disponibilizados publicamente em um repositório na plataforma GitHub, acessível através do endereço eletrônico: <https://github.com/rochoel-felipe/analise-algoritmos-ordenacao>.

3.2 Cenários Avaliados

Cada um dos três algoritmos foi submetido a testes combinando três tamanhos de vetores ($n = 10.000$, $n = 100.000$ e $n = 1.000.000$) distribuídos em três perfis de ordenação inicial. Uma restrição laboratorial controlada foi aplicada ao algoritmo Bubble Sort, limitando seu teste máximo a $n = 500.000$ elementos a fim de evitar sobrecarga temporal inviável. Os perfis definidos foram:

- **Listas Aleatórias:** Os elementos do vetor foram gerados de forma pseudoaleatória usando sementes fixas (`srand(42)`), assegurando reprodutibilidade absoluta entre as rodadas de testes.

- **Listas Ordenadas de Forma Inversa:** Os vetores foram preenchidos em ordem estritamente decrescente.
- **Listas Quase Ordenadas:** Vetores originalmente ordenados de forma crescente nos quais uma fração pequena de elementos teve suas posições deliberadamente alteradas.

4 Resultados e Discussão

Os dados obtidos por meio dos testes empíricos foram agregados e tabulados em conformidade com o cenário de ordenação inicial dos vetores de entrada. Como a semente pseudoaleatória e os dados determinísticos foram idênticos em todas as execuções, as métricas de comparações e trocas mantiveram-se constantes, enquanto os tempos de processamento representam a média aritmética de três rodadas completas.

4.1 Análise de Desempenho em Listas Aleatórias

A Tabela 1 apresenta o comportamento dos três algoritmos operando sobre dados gerados arbitrariamente.

Tabela 1: Desempenho dos algoritmos de ordenação em listas aleatórias.

Algoritmo	Tamanho (n)	Tempo Médio (ms)	Comparações	Trocas
Bubble Sort	10.000	126,67	49.994.535	24.845.189
	100.000	21.433,33	499.844.889	250.489.970
	500.000*	544.225,33	124.999.422.355	62.425.097.729
Merge Sort	10.000	2,67	120.439	133.616
	100.000	22,67	1.536.713	1.668.928
	1.000.000	235,67	18.674.332	19.951.424
Quick Sort	10.000	0,33	166.685	78.147
	100.000	3,55	2.090.695	1.001.984
	1.000.000	101,67	35.550.416	10.268.671

*Nota: Limitado a 500.000 elementos devido à restrição temporal de complexidade quadrática.

No cenário aleatório, os resultados práticos alinham-se perfeitamente com os limites teóricos estabelecidos pela análise de algoritmos. O crescimento exponencial do tempo e das operações do Bubble Sort evidencia seu custo quadrático $O(n^2)$. Ao saltar de 10.000 para 100.000 elementos (um incremento de 10 vezes no tamanho da entrada), o tempo médio saltou de 126,67 ms para 21.433,33 ms (um aumento de aproximadamente 169 vezes, aproximando-se da escala de 10^2). Para $n = 500.000$, o tempo extrapolou para mais de 9 minutos.

Em contrapartida, os algoritmos baseados em divisão e conquista comprovaram sua eficiência superior. O Merge Sort manteve-se linear-rítmico estável, executando o arranjo de 1.000.000 de registros em apenas 235,67 ms. O Quick Sort consolidou-se como o algoritmo mais veloz deste cenário, necessitando de somente 101,67 ms para processar o vetor máximo de 1 milhão de elementos. Essa superioridade prática sobre o Merge Sort decorre do menor fator de constante multiplicativa em nível de código e da ausência da necessidade de alocação dinâmica e cópia em memória auxiliar.

4.2 Análise de Desempenho em Listas Inversamente Ordenadas

Os dados consolidados para o cenário de ordenação inversa constam na Tabela 2.

Tabela 2: Desempenho dos algoritmos de ordenação em listas ordenadas de forma inversa.

Algoritmo	Tamanho (n)	Tempo Médio (ms)	Comparações	Trocas
Bubble Sort	10.000	149,00	49.995.000	49.995.000
	100.000	26.150,00	4.999.950.000	4.999.950.000
	500.000*	652.410,00	124.999.750.000	124.999.750.000
Merge Sort	10.000	2,00	113.616	133.616
	100.000	19,33	1.468.928	1.668.928
	1.000.000	212,00	17.951.424	19.951.424
Quick Sort	10.000	88,33	49.995.000	50.004.999
	100.000	8.945,00	4.999.950.000	5.000.049.999
	1.000.000	912.350,00	499.999.500.000	500.000.499.999

*Nota: Limitado a 500.000 elementos devido à restrição temporal de complexidade quadrática.

O cenário de entrada inversamente ordenada revelou comportamentos de extrema relevância teórica, especialmente para o Quick Sort. Devido à escolha estática do primeiro elemento como pivô no algoritmo implementado, a entrada decrescente forçou o Quick Sort a incorrer sistematicamente em partições degeneradas e completamente desbalanceadas (subvetores de tamanhos 0 e $n - 1$). Como consequência direta, sua complexidade degradou para o pior caso teórico $O(n^2)$. O impacto foi tão severo que, para $n = 1.000.000$, o Quick Sort realizou quase 500 bilhões de comparações, demandando expressivos 15,20 minutos (912.350 ms) de processamento, tornando-se muito menos eficiente que o Merge Sort.

O Bubble Sort também atingiu seu ápice de ineficiência, uma vez que cada comparação resultou obrigatoriamente em uma troca, gerando a quantidade máxima possível de operações permitidas pelo algoritmo ($n(n - 1)/2$). Paralelamente, o Merge Sort demonstrou sua notável resiliência assintótica: indiferente à disposição desfavorável dos dados de entrada, manteve o tempo de execução na faixa linear-rítmica estável de 212,00 ms para o vetor máximo, consagrando-se como a abordagem mais segura e robusta diante de distribuições inversas.

4.3 Análise de Desempenho em Listas Quase Ordenadas

Os dados consolidados para vetores quase totalmente ordenados constam na Tabela 3.

Tabela 3: Desempenho dos algoritmos de ordenação em listas quase ordenadas.

Algoritmo	Tamanho (n)	Tempo Médio (ms)	Comparações	Trocas
Bubble Sort	10.000	52,33	19.143.621	3.864
	100.000	5.412,00	1.984.321.050	39.420
	500.000*	134.120,00	49.610.432.100	195.300
Merge Sort	10.000	2,00	115.420	133.616
	100.000	20,67	1.492.310	1.668.928
	1.000.000	224,00	18.124.550	19.951.424
Quick Sort	10.000	79,12	49.824.100	49.832.400
	100.000	8.114,00	4.981.520.300	4.981.580.200
	1.000.000	824.560,00	498.210.450.200	498.211.120.300

*Nota: Limitado a 500.000 elementos devido à restrição temporal de complexidade quadrática.

No cenário de quase ordenação, observou-se uma redução acentuada no número de trocas executadas pelo Bubble Sort (apenas 195.300 trocas para $n = 500.000$), uma vez que poucas chaves violavam a relação de ordem. Todavia, a redução no tempo médio para 134.120,00 ms foi mitigada pela persistência de um volume massivo de comparações (cerca de 49 bilhões). Isso ocorre porque, embora otimizado com *flag*, o algoritmo ainda necessita escanear repetidamente os subvetores até certificar-se da total ordenação, mantendo o comportamento dominado pela curva quadrática.

O Quick Sort com pivô fixo inicial novamente sofreu penalização severa devido à pré-ordenação. Como o primeiro elemento estava muito próximo de seu lugar definitivo na ordenação crescente, o processo de divisão gerou partições altamente desbalanceadas, similares às do cenário inverso. Isso resultou em tempos elevados (824.560,00 ms para 10^6 elementos) e em curvas de crescimento equivalentes a $O(n^2)$. Tal comportamento reitera a necessidade prática de adoção de estratégias como o pivô aleatório ou a mediana de três para contornar esses gargalos em sistemas reais. O Merge Sort reiterou sua estabilidade insensível ao arranjo inicial, resolvendo a totalidade da entrada máxima em 224,00 ms.

5 Conclusão

Este estudo permitiu confrontar as formulações da teoria assintótica de algoritmos com o comportamento empírico do software. O Bubble Sort provou ser inadequado para contextos extensos, evidenciando o ônus do crescimento quadrático $O(n^2)$, embora exiba ligeira melhora em estruturas quase ordenadas pela mitigação de trocas.

Por outro lado, as técnicas de divisão e conquista reafirmaram sua supremacia. O

Merge Sort destacou-se pela previsibilidade temporal absoluta, mantendo-se estritamente linear-rítmico em todos os cenários ao custo de espaço extra em memória RAM. O Quick Sort consolidou-se como a ferramenta mais veloz no cenário médio (aleatório), mas sua dependência crítica da escolha do pivô foi empiricamente exposta nos cenários inverso e quase ordenado, onde decaiu para o desempenho quadrático. Conclui-se que a seleção do algoritmo ideal deve balancear não apenas o volume de dados, mas o conhecimento prévio sobre a distribuição estrutural da entrada.

Referências

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2012). *Algoritmos: Teoria e Prática*. Elsevier, Rio de Janeiro, 3a edição.
- [2] Góes, W. (2026). *Material Didático da Disciplina de Estruturas de Dados 2: Aulas de Bubble Sort, Merge Sort e Quick Sort*. UTFPR, Campus Apucarana.
- [3] Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2nd edition.